REPORT

All experiments were carried out on the university's linux20 and Linux15 machines, when there was no other user logged in

Number of cores : 4

Processor : Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz

OS : NAME="Ubuntu" VERSION="20.04.6 LTS (Focal Fossa)"

Compiler Version : 9.4.0

Exercise 2.1

Times with openMP

Threads	$n = 10^4$				
	Initialization (s)	Serial (s)	Parallel (s)	Speedup	Efficiency (%)
1	0.0005	0.3455	0.3873	0.3492	0.9900
2	0.0007	0.3537	0.1804	1.9600	98.010
4	0.0006	0.3454	0.0983	3.5160	87.844
8	0.0008	0.3493	0.1000	3.4940	43.684

Threads	$n = 10^5$				
	Initialization (s)	Serial (s)	Parallel (s)	Speedup	Efficiency (%)
1	0.0041	34.032	34.679	0.9800	98.134
2	0.0071	34.013	17.876	1.9000	95.132
4	0.0042	34.117	9.7804	3.4880	87.208
8	0.0072	34.076	9.6007	3.5500	44.368

Times with pthreads

Threads	$n = 10^4$				
	Initialization (s)	Serial (s)	Parallel (s)	Speedup	Efficiency (%)
1	0.0004	0.3766	0.3873	0.9766	97.276
2	0.0004	0.3787	0.1870	2.0233	101.23
4	0.0004	0.3788	0.1667	2.2733	56.843
8	0.0004	0.3750	0.1461	2.5700	32.150

Threads	$n = 10^5$				
	Initialization (s)	Serial (s)	Parallel (s)	Speedup	Efficiency (%)
1	0.0044	34.012	38.194	0.9866	98.623
2	0.0065	34.080	17.871	1.9066	95.346
4	0.0072	34.051	13.627	2.5000	62.466
8	0.0104	34.069	10.688	3.1900	39.880

For the problem with polynomial multiplication, exactly the same logic was followed as with pthreads. An interesting observation that we did not put in the arrays, because the differences in time were small, is that the times were better with the use of guided and dynamic scheduling where for 4 and 8 threads (n = = 10^5) they had a better time by 2-3 seconds than static scheduling. This makes sense since for a greater degree of polynomials, the workload is larger and therefore the distribution is not uniform, therefore We want it to be possible when a thread finishes to be able to get work immediately. Note that for static,1 or for static,10 i.e. to take each thread in a circle the performance is similar to that of dynamic-
guided. The problem existed only in plain static. In the end, the schedule was kept guided. Regarding the times of the array we see logical improvements and speedup in both cases. The times for 1,2,4 threads are a little faster than with Pthreads but for 8 threads we see that the time in one case increases slightly and in the other it decreases slightly, i.e. there is no noticeable reduction from 4->8 threads which on the one hand is reasonable since our machine has 4 cores without hyperthreading. On the other hand, in Pthreads we saw a noticeable improvement there as well, perhaps due to time concealment
access to memory. After a survey we found that openMP many
times I can if I ask for more threads from the physical cores to give as many

are the Physicals.
“Normally, the OpenMP runtime automatically chooses the thread count such that all cores on your system receive one thread and the system is not overloaded … OpenMP does not differentiate physical cores and logical cores available with Hyper-Threading.”

https://www.openmp.org/spec-html/5.0/openmpse51.html

Then we set the environment variable OMP_NUM_THREADS to
A larger number as reported in the link but there was no change.

Exercise 2.2

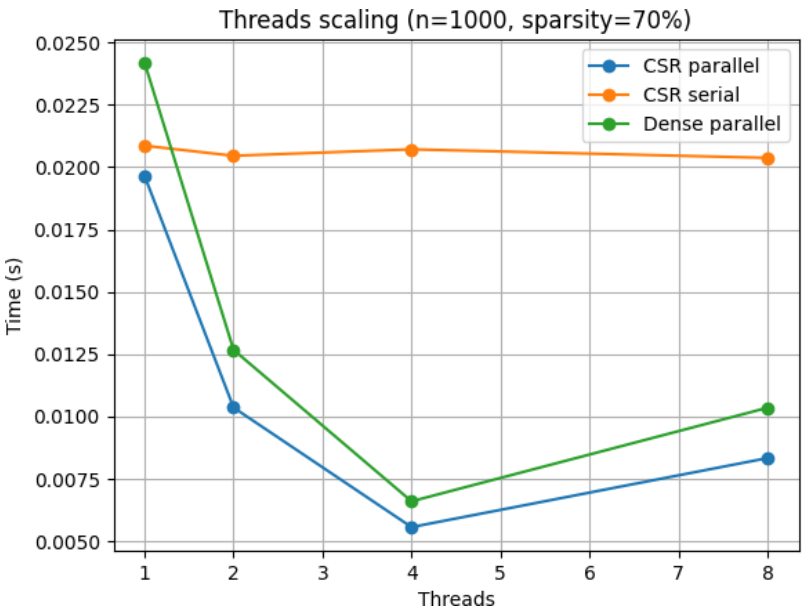
The problem of multiplying arrays by vector, iteratively (the result vector becomes the input vector in the next iteration) was solved as follows:

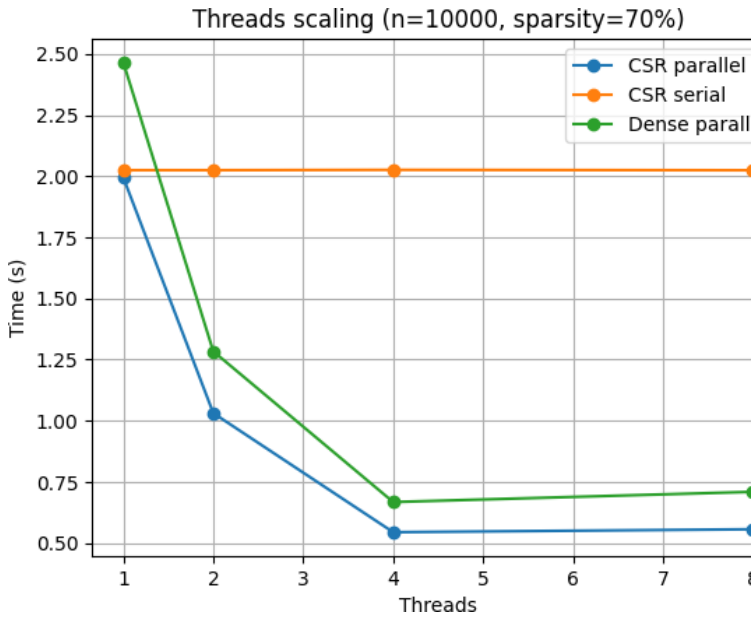
First, the array and the vector are initialized with random numbers. Then the CSR structures are initialized in a parallel way, but there is a serialization point, when we need to measure how many non-zero elements there are up to that line (dependency on the previous line).
We measure in parallel how many elements each line has, since there is no dependence between different lines, and all that is done serially is a for as long as the dimension is.

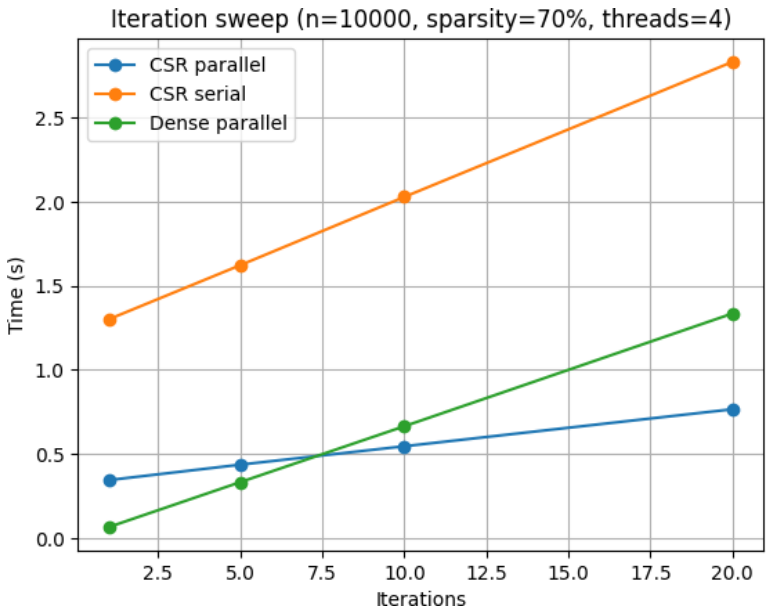
One change we made was that we kept a counter for how many non-zero elements there are, but they are counted in the random initialization of the array at the beginning, so it basically didn't count in the initialization time of the csr structures. We finally put it further down in a parallelized for.

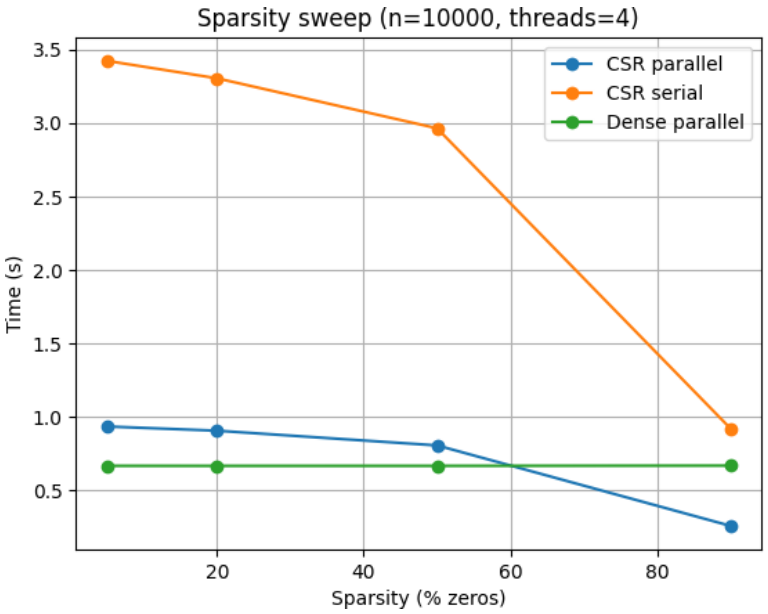
Another test we did was in the creation of the CSR arrays (having calculated the array row_indeces) to check if we have reached the exact zero elements (row_indeces[i + 1] - row_indeces[i]) so that it breaks the loop not to continue since there are no other zero elements in this series. However, there was a small burden on time and
removed (logically because the control in relation to how many loops we saved was not effective in time).

Below are the diagrams, which ran for 4 times each experiment and with fixed 10000 rows and columns each array and 10 repetitions (to be relatively heavy so the times are more obvious) with 4 threads to take advantage of the parallelism and zeros 70%.









<u>Comparing Parallel CSR with Serial :</u>

Initially, we see that as the threads increase (up to 4 which are also the physical cores) the parallel implementation does extremely well since it takes full advantage of the parallelism in both initialization and multiplication execution.

Then we see that for 8 threads there is a small overhead which, however, at n=10000 is hidden since the workload is higher and up to 2 threads per core can improve the performance a little due to hiding time in ram which is equal to the time of the context switch. Also

We see that for both many and few iterations the serial-parallel time difference is constant. An interesting observation is that for a very large number of zeros (90%) the difference between parallel and serial CSR decreases sharply and this is explained by the fact that the more zeros the less work is done by the CSR algorithm (there is no multiplication –
accesses to memory).

Comparison of parallel CSR with dense representation parallel:

We see that the parallel dense representation algorithm does just as well in some cases and even for less sparse arrays it is better, e.g. 40-50%, which is logical since the csr method takes advantage of the many zeros and in fact we see that for more zeros it is better. For a fixed number of zeros and repetitions, the graphic
The dense representation is shifted vertically – i.e. for each thread number it is correspondingly worse. Similarly, for many iterations, the CSR method is better since the operations it performs are
less and this in the many repetitions seems more intense (that is, that it deserves the overhead of the creation in terms of the work that in the end the method spares us).

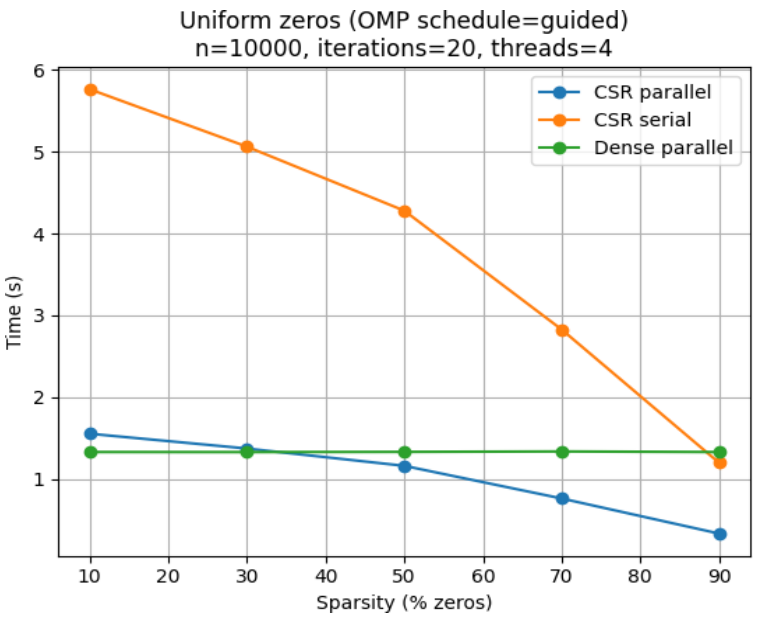
Unequal Distribution 0:

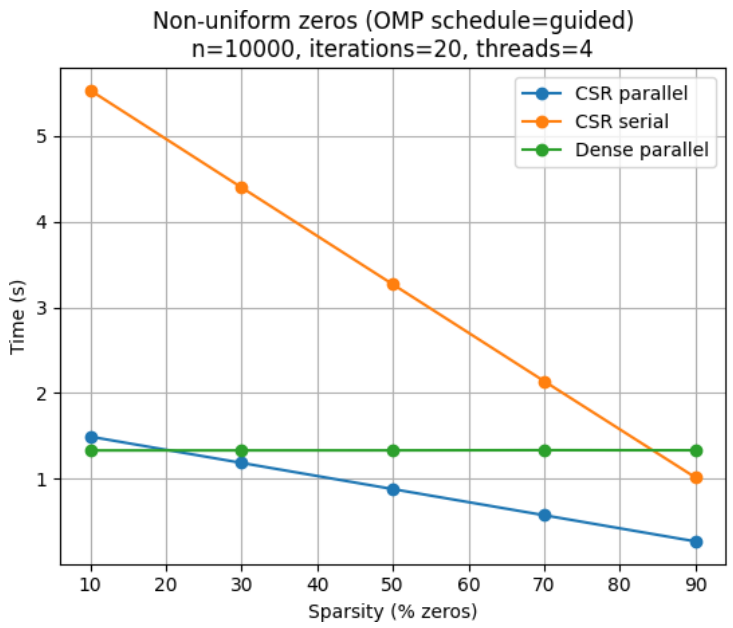
With the parameter Y(yes) in the program, it puts all the zeros in the array (as many as they are according to the percentage) at the beginning of the array. Below are the diagrams for uneven and normal distribution of zeros with static and dynamic scheduling respectively.

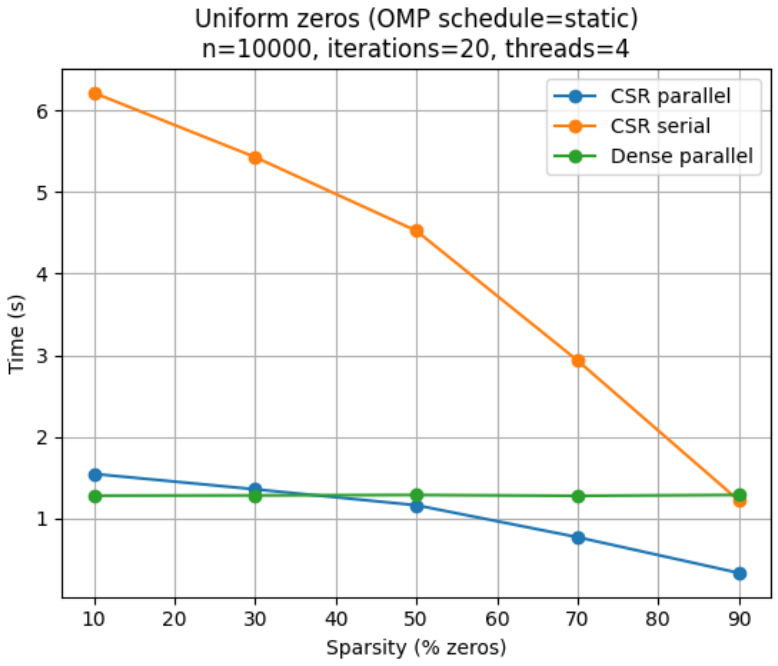
Attention: The scheduling in the program is set as a runtime so that it can be changed by the environment variable. If you run the program through the script then there is no problem because it sets the variable appropriately.

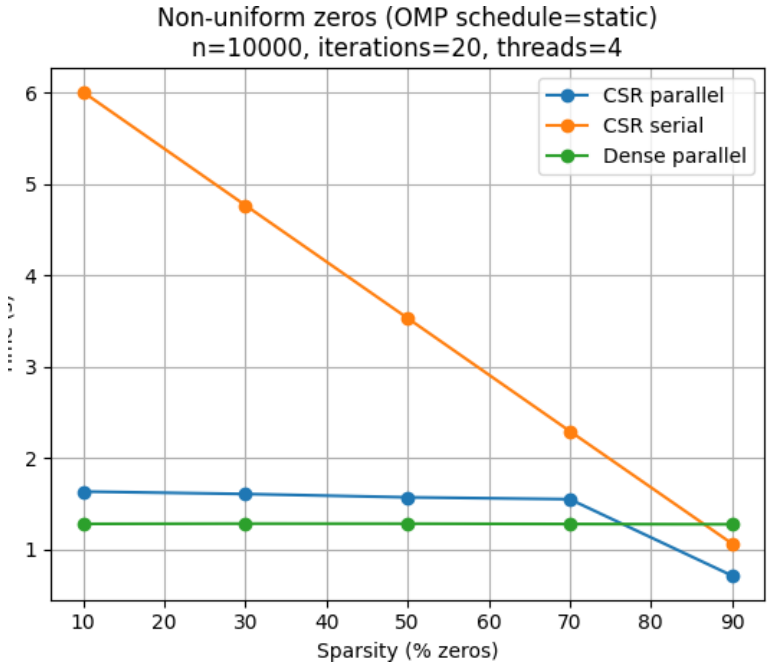
However, if it is run manually, the user is responsible for having correctly defined the environment variable (the measurements have been made with static).

As we see in the diagrams for uniform distribution there is no difference between static and guided, but when it is non-uniform the parallel with guided works much better (especially for 70% zeros we see a 50% improvement).
This is because when the distribution is uneven, some threads that only have zeros do not actually work because all the lines they have taken are filled with zeros. So when we have static and the board just breaks into 4
tracks will not all threads have the same work, which is fixed with guided (or even static,2 e.g. where the times are the same as guided and not
the diagrams were inserted for this reason).









Exercise 2.3

This problem highlights the Speedup of a classification algorithm by parallelizing the program with the use of OpenMP and in particular with the use of tasks. The merge-sort algorithm is implemented both serially and along with the difference of only a few commands. The point is to initialize a array of the desired size in a deterministic way, so that for a fixed size the array values do not change. In the
Then, this array is classified with the merge-sort classification whereby the array is subdivided each time until it reaches a

individual element and retrospectively when the arrays are merged, the elements are sorted.

Similar to the code of the slides for the Fibonacci problem, a single thread must initiate the recursive call of the algorithm that concerns the Part of the array subdivision and that creates the initial tasks. After the creation of these tasks, a taskwait must be made in order to The merger will take place only after the two sub-arrays have been prepared.

The critical point in this problem is the selection of a minimum array size for which the parallel algorithm will create additional tasks. In short, we need to define a condition through the if() clause, where for more than a certain size of arrays tasks will be created that provide the ability to perform parallel tasks. On the other hand, if the The size of the array does not exceed this number, the code piece of the task is executed serially.

The minimum array size for which tasks are created is set to 10000. This number was chosen experimentally since for smaller numbers, the execution time of the parallel algorithm exceeds that of the serial algorithm. This is because the overhead for creating and implementing tasks is very large and not worth it for small array sizes that can be sorted quickly and serially.

The results are presented below, while the times given are derived from the average of 5 executions.

<u>Array Size: 10^7</u>

Algorithm	**Thread count**			
	1	2	4	8
Parallel	2.390870 seconds	1.279136 seconds	.719918 seconds	.790530 seconds
Serial	2.035940 seconds			

Speedup S = Tserial / Tparallel

	Thread count			
	1	2	4	8
S	0.86	1.6	2.75	2.71

Array Size: 10^8

Algorithm	Thread count			
	1	2	4	8
Parallel	24.530468 seconds	13.229354 seconds	7.452214 seconds	7.809406 seconds
Serial	21.412444 seconds			

Speedup

	Thread count			
	1	2	4	8
S	0.87	1.63	2.87	2.81

As mentioned earlier, the Speedup of the program was only achieved by **limiting task creation** for recursive calls regardless of array size. This significantly speeds up the parallel algorithm as evidenced by the results, since for "small" sizes it is executed serially and for **large-scale parallel execution.** When running with a **single thread**, he execution time is slightly higher **than the serial version** since in reality we don't have a parallel but only an overhead to
this thread is created. In addition, with the increase in threads, the execution time as expected since, multiple threads can perform different tasks in parallel. Obviously, from 4 to 8 threads there is no improvement, since the number of cores is half and

Threads can't actually perform their work completely at once because of context switches. The speedup remains similar for both board sizes, with 10^8 showing a minimum
improvement.